P0217R1
Jens Maurer <Jens.Maurer@gmx.net>
Audience: Evolution and Core Working Groups
2016-03-20

P0217R1: Proposed wording for structured bindings

Introduction

This paper presents the proposed wording to implement structured bindings as described by Herb Sutter, Bjarne Stroustrup, and Gabriel dos Reis in P0144R2. That paper incorporates feedback from the EWG session in Jacksonville as follows:

- change to [] syntax
- introduced variables are, in all cases, references to the value of the initializer
- the cv-qualifiers and ref-qualifier of the decomposition declaration are applied to the reference introduced for the initializer, not for the individual member aliases
- support bit-fields
- use tuple_size and tuple_element

In this paper, changes relative to the predecessor P0217R0 are highlighted with blue text. The specifications for a decomposition declaration in section 7.1.6.4 have been rewritten almost entirely and are not so marked.

Wording

Change in 5.1.1 [expr.prim] paragraph 8:

> ... The type of the expression is the type of the identifier. The result is the entity denoted by the identifier. The result is the entity denoted by the identifier. The result is an lvalue if the entity is a function, variable, or data member and a prvalue otherwise. **The result is a bit-field if the identifier designates a bit-field (7.1.6.4 [dcl.spec.auto]).**

Change in 5.2.5 [expr.ref] paragraph 3:

> Abbreviating *postfix-expression.id-expression*x as E1.E2, E1 is called the object expression. **If `E2` is a bit-field, `E1.E2` is a bit-field.** ...

In section 6.5 [stmt.iter] paragraph 1, change the grammar to allow a decomposition declaration:

> *for-range-declaration:*
> *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq declarator*
> ***attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ `[` *identifier-list* `]`**

In section 7 [dcl.dcl] paragraph 1, change the grammar to allow a decomposition declaration:

> *simple-declaration:*
> *decl-specifier-seq init-declarator-list*$_{opt}$ `;`
> *attribute-specifier-seq decl-specifier-seq init-declarator-list* `;`
> ***attribute-specifier-seq*$_{opt}$ *decl-specifier-seq ref-qualifier*$_{opt}$ `[` *identifier-list* `]` *brace-or-equal-initializer* `;`**

Add a new paragraph after 7 [dcl.dcl] paragraph 8:

> **A *simple-declaration* with an *identifier-list* is called a *decomposition declaration*. The *decl-specifier-seq* shall contain only the *type-specifier* `auto` (7.1.6.4 [dcl.spec.auto]) and *cv-qualifier*s. The *brace-or-equal-initializer* shall be of the form "= *assignment-expression*" or of the form "{ *assignment-expression* }", where the *assignment-expression* is of array or non-union class type.**

Change in 7.1.6.2 [dcl.type.simple] paragraph 4:

> For an expression e, the type denoted by `decltype(e)` is defined as follows:
>
> - **if e is an unparenthesized *id-expression* naming an lvalue or reference introduced from the *identifier-list* of a decomposition declaration, `decltype(e)` is the referenced type as given in the specification of the decomposition declaration (7.1.6.4 [dcl.spec.auto]);**
> - **otherwise,** if e is an unparenthesized id-expression or an unparenthesized class member access (5.2.5), decltype(e) is the type of the entity named by e. If there is no such entity, or if e names a set of overloaded functions, the program is ill-formed;

- otherwise, if e is an xvalue, decltype(e) is T&&, where T is the type of e;
- otherwise, if e is an lvalue, decltype(e) is T&, where T is the type of e;
- otherwise, decltype(e) is the type of e.

Add after 7.1.6.4 [dcl.spec.auto] paragraph 8:

A decomposition declaration introduces the *identifier*s of the *identifier-list* as names in the order of appearance, where v_i denotes the *i*-th identifier, with numbering starting at 1. Let *cv* denote the *cv-qualifier*s in the *decl-specifier-seq*. First, a variable with a unique name *e* is introduced. If the *assignment-expression* in the *brace-or-equal-initializer* has array type A and no *ref-qualifier* is present, `e` has type *cv* `A` and is copy-initialized or direct-initialized from the *assignment-expression* as specified by the form of the *brace-or-equal-initializer*. Otherwise, `e` is defined as-if by

attribute-specifier-seq$_{\texttt{opt}}$ *decl-specifier-seq ref-qualifier*$_{\texttt{opt}}$ *e brace-or-equal-initializer* ;

where the parts of the declaration other than the *declarator-id* are taken from the corresponding decomposition declaration. The type of the expression `e` is called `E`. [Note: `E` is never a reference type (Clause 5 [expr]). -- end note]

If `E` is an array type with element type T, the number of elements in the *identifier-list* shall be equal to the number of elements of `E`. Each v_i is the name of an lvalue that refers to the element *i*-1 of the array and whose type is T; the referenced type is T. [Note: The top-level cv-qualifiers of T are *cv*. -- end note]

Otherwise, if the expression `std::tuple_size<E>::value` is a well-formed integral constant expression, the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (3.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the initializer is *e*`.get<`*i*`-1>()`. Otherwise, the initializer is `get<`*i*`-1>(`*e*`)`, where `get` is looked up in the associated namespaces (3.4.2 [basic.lookup.argdep]). [Note: Ordinary unqualified lookup (3.4.1 [basic.lookup.unqual]) is not performed. -- end note] In either case, `e` is an lvalue if the type of the entity `e` is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<`*i*`-1,E>::type`, each v_i is a variable of type "reference to T_i" initialized with the initializer, where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise; the referenced type is T_i.

Otherwise, all of `E`'s non-static data members and bit-fields shall be public direct members of `E` or of the same unambiguous public base class of `E`, `E` shall not have an anonymous union member, and the number of elements in the *identifier-list* shall be equal to the number of non-static data members of `E`. The *i*-th non-static data member of E in declaration order is designated by m_i. Each v_i is the name of an lvalue that refers to the member m_i of `e` and whose type is *cv* T_i, where T_i is the declared type of that member; the referenced type is *cv* T_i. The lvalue is a bit-field if that member is a bit-field. [Example:

```
struct S { int x; volatile double y; };
S f();
const auto [ x, y ] = f();
```

The type of the expression `x` is "const int", the type of the expression `y` is "const volatile double". -- end example]